Guía del Proyecto N°1 (Control y supervisión de sistema secuencial electroneumático). Explica cómo preparar la PC del laboratorio para ejecutar el HMI: los dos intérpretes de Python, las bibliotecas de Qt, el editor Qt Design Studio y la conexión OPC DA con RSLinx. La descarga del proyecto desde GitHub y el uso de los lanzadores `.bat` *están en la Guía 2 (*`guia_replicar_proyecto_github.pdf`*). La biblioteca* `gbda_aut.dll` *que se registra en la sección IV está en esta misma carpeta (*`instalacion_python`*).*

Índice

I. Qué se instala y por qué **2**
I.1. Requisitos de la PC . . . 3

II. Instalación de Python **3**
II.1. Descargar los instaladores . . . 3
II.2. Opciones del instalador . . . 3
II.3. Comprobar los dos intérpretes . . . 4
II.4. Entornos virtuales . . . 4
II.5. Dependencias . . . 4
II.6. Instalación sin internet . . . 5

III. Qt: PySide6 y Qt Design Studio **6**
III.1. PySide6 en el entorno del HMI . . . 6
III.2. Qt Designer o Qt Design Studio . . . 6
III.3. Instalar Qt Design Studio (opcional) . . . 6
III.4. Abrir el HMI en Qt Design Studio . . . 7
III.5. Prueba de la pantalla sin PLC . . . 8

IV. Conexión OPC DA con RSLinx **9**
IV.1. La cadena de comunicación . . . 9
IV.2. Qué es `gbda_aut.dll` y por qué hace falta . . . 9
IV.3. Origen de la DLL incluida en esta carpeta . . . 9
IV.4. Registrar la DLL . . . 10
IV.5. Comprobar el registro desde Python . . . 10
IV.6. Preparar RSLinx . . . 11
IV.7. Primera lectura con OpenOPC . . . 11
IV.8. Diagnóstico con la herramienta del repositorio . . . 12
IV.9. Cómo usa la conexión el gateway . . . 12
IV.10. Conectar el HMI en modo REAL . . . 13

V. Problemas típicos **14**

VI. Lista de pruebas de la instalación **15**

DQ P1

I. Qué se instala y por qué

El sistema de supervisión son dos programas en Python que corren en la misma PC y se comunican por ZeroMQ (Figura 1):

- El **gateway** (`gateway/`) es el único que habla con el PLC. Lee y escribe por OPC DA a través de RSLinx Classic y publica el estado unas diez veces por segundo.
- El **HMI** (`ui/`) es la pantalla: sinóptico de cilindros, GRAFCET, mandos, parámetros, cronograma, monitor de E/S, registro y latencia. Está escrito en QML (Qt Quick) y se ejecuta con PySide6.

Cada uno necesita un intérprete distinto. La conexión con RSLinx pasa por `gbda_aut.dll`, una DLL COM *en proceso* de 32 bits: Windows sólo puede cargarla dentro de un proceso de 32 bits, así que el gateway corre con **Python 3.12 de 32 bits**. PySide6, en cambio, sólo se publica para 64 bits, así que el HMI corre con **Python 3.13 (o 3.12) de 64 bits**. Cada intérprete tiene su propio entorno virtual dentro del repositorio: `.venv32` y `.venv`.

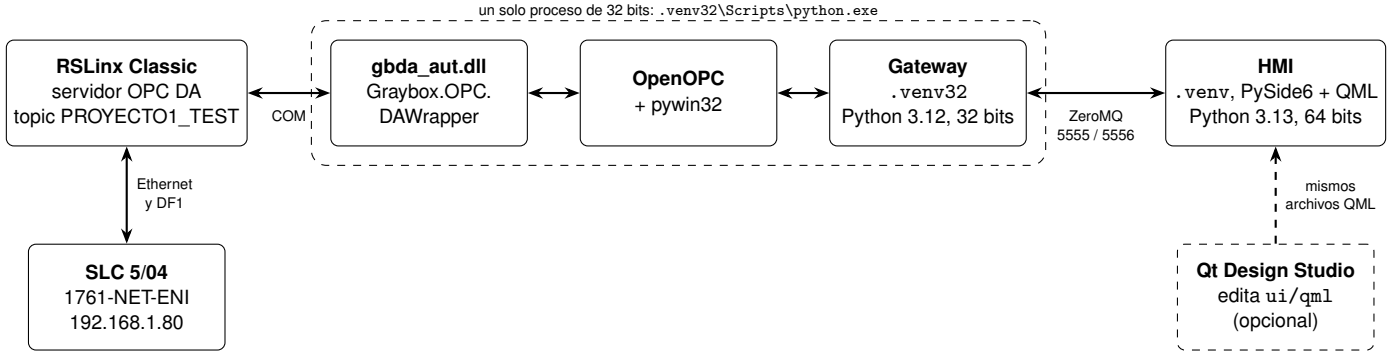


Figura 1: Qué se instala en la PC del laboratorio y cómo se conecta. Lo que está dentro del recuadro punteado se carga en el mismo proceso de 32 bits.

Componente	Intérprete	Entorno	Paquetes principales	Dónde se explica
Gateway OPC	Python 3.12, 32 bits	`.venv32`	pyzmq, msgpack, loguru, pywin32, OpenOPC-Python3x	§II, §IV
HMI, puente, gemelo local y pruebas	Python 3.13 (o 3.12), 64 bits	`.venv`	PySide6, pyzmq, msgpack, loguru, aiohttp, paho-mqtt, pytest	§II, §III
`gbda_aut.dll`	— (se registra en Windows)	—	Graybox OPC DA Automation Wrapper, copia de OpenOPC	§IV
Qt Design Studio	— (programa aparte)	—	Editor visual de QML; opcional, no hace falta para ejecutar el HMI	§III.3

Tabla 1: Componentes de software de la PC del laboratorio.

DQ DQ P1 P1

I.1. Requisitos de la PC

Requisito	Detalle
Windows 10 u 11 de 64 bits	En Windows de 64 bits conviven las carpetas `System32` (64 bits) y `SysWOW64` (32 bits); la DLL va en la segunda (§IV.4).
Permisos de administrador	Sólo para copiar y registrar `gbda_aut.dll`. Python, el repositorio y los entornos virtuales se instalan con la cuenta normal.
RSLinx Classic con OPC	Edición Gateway, OEM o Single Node. La edición *Lite* no trae servidor OPC. En el laboratorio ya está instalado con el driver `AB_ETH-2` y el topic `PROYECTO1_TEST`.
Internet	Para descargar Python y los paquetes de `pip`. Sin internet se instalan ruedas descargadas en otra PC (§II.6).
Repositorio del HMI	Descargado en una carpeta corta, sin espacios ni acentos, por ejemplo `C:\Proyectos\HMI` (Guía 2, [17]).

Tabla 2: Requisitos previos.

Orden recomendado: (1) instalar los dos Python (§II); (2) descargar el repositorio y ejecutar `lab\instalar.bat` (Guía 2), que crea los entornos e instala los paquetes; (3) registrar `gbda_aut.dll` (§IV.4); (4) comprobar RSLinx y la conexión OPC (§IV); (5) correr la lista de pruebas (§VI). Qt Design Studio es opcional y sólo se instala en la PC donde se edite la pantalla.

II. Instalación de Python

II.1. Descargar los instaladores

Los dos instaladores se descargan de la página oficial de versiones para Windows [1]:

Intérprete	Archivo	En la página de python.org	Uso
Python 3.12, 32 bits	`python-3.12.x.exe` (sin `amd64`)	Enlace *Windows installer (32-bit)* de la versión 3.12 más reciente que tenga instalador	Gateway OPC (`.venv32`)
Python 3.13, 64 bits	`python-3.13.x-amd64.exe`	Enlace *Windows installer (64-bit)*	HMI, puente, gemelo y pruebas (`.venv`)

Tabla 3: Instaladores de Python.

Las versiones de Python que ya sólo reciben correcciones de seguridad se publican sin instalador para Windows; por eso hay que elegir la última 3.12 que lo tenga. Si sólo se consigue Python 3.12 de 64 bits para el HMI, también sirve: `lab\instalar.bat` busca primero 3.13 y después 3.12 de 64 bits.

II.2. Opciones del instalador

Instala primero el de 32 bits y después el de 64 bits, con las mismas opciones:

1. En la primera pantalla marca *Use admin privileges when installing py.exe* si tienes permisos de administrador. La casilla *Add python.exe to PATH* puede quedar sin marcar: los lanzadores del proyecto no usan el `python` del PATH, sino el lanzador `py` y los entornos virtuales. Si la marcas, hazlo sólo en uno de los dos para que `python` no sea ambiguo.

2. Pulsa *Customize installation*. En *Optional Features* deja marcados *pip*, *tcl/tk and IDLE* (el HMI de respaldo v6.1, en `old/`, usa Tkinter) y, sobre todo, ***py launcher***. Sin el lanzador `py`, `instalar.bat` no puede distinguir entre los dos intérpretes.
3. En *Advanced Options* deja la carpeta propuesta. Para la instalación por usuario es `%LOCALAPPDATA%\Programs\Python\Python312-32` (32 bits) y `...\Python313` (64 bits).
4. Al terminar, si aparece *Disable path length limit*, púlsalo (pide administrador). Evita errores con rutas largas dentro de los entornos virtuales.

II.3. Comprobar los dos intérpretes

Abre una consola **nueva** (Símbolo del sistema o PowerShell), para que lea el PATH actualizado, y lista los intérpretes que conoce el lanzador [2]:

Listado 1: Intérpretes instalados según el lanzador `py`.

```
C:\> py -0p
 -V:3.13 *        C:\Users\<usuario>\AppData\Local\Programs\Python\Python313\python.exe
 -V:3.12-32       C:\Users\<usuario>\AppData\Local\Programs\Python\Python312-32\python.exe
```

Deben aparecer las dos líneas; el asterisco marca el intérprete por defecto. La arquitectura se confirma así (las dos órdenes deben imprimir `True`):

Listado 2: Comprobación de 32 y 64 bits.

```
py -3.12-32 -c "import sys; print(sys.version, sys.maxsize < 2**32)"
py -3.13-64 -c "import sys; print(sys.version, sys.maxsize > 2**32)"
```

Si `py` no se reconoce, falta el *py launcher*: vuelve a ejecutar el instalador, elige *Modify* y márcalo.

II.4. Entornos virtuales

Un entorno virtual es una carpeta con una copia aislada del intérprete y sus paquetes [3]. El proyecto usa dos, en la raíz del repositorio, y nunca instala paquetes en el Python del sistema. Los crea `lab\instalar.bat` (Guía 2); a mano, lo equivalente es:

Listado 3: Creación manual de los entornos (lo mismo que hace `lab\instalar.bat`).

```
cd /d C:\Proyectos\HMI
rem Gateway: Python 3.12 de 32 bits
py -3.12-32 -m venv .venv32
.venv32\Scripts\python.exe -m pip install --upgrade pip
.venv32\Scripts\python.exe -m pip install -r requirements-gateway.txt
rem HMI, puente, gemelo y pruebas: Python de 64 bits
py -3.13-64 -m venv .venv
.venv\Scripts\python.exe -m pip install --upgrade pip
.venv\Scripts\python.exe -m pip install -r requirements-hmi.txt "pytest>=7,<9"
```

No hace falta *activar* los entornos: basta con llamar al `python.exe` de cada uno por su ruta. Así lo hacen todos los lanzadores, y por eso un `.bat` nunca ejecuta el gateway con el Python equivocado.

II.5. Dependencias

Listado 4: `requirements-gateway.txt`: dependencias del gateway (32 bits).

```
pyzmq>=26,<27
msgpack>=1.0,<2
```

```
loguru>=0.7,<0.8
# Solo Windows, solo EMULATE / REAL:
pywin32>=306; sys_platform == "win32"
OpenOPC-Python3x>=1.3; sys_platform == "win32"
```

Paquete	Entorno	Para qué
pyzmq, msgpack	los dos	Mensajes entre gateway y HMI: el gateway publica el estado en `tcp://127.0.0.1:5555` y recibe órdenes en `5556`.
loguru	los dos	Registro de eventos en consola y archivo.
pywin32	`.venv32`	Acceso a COM desde Python (`win32com`, `pythoncom`) [4].
OpenOPC-Python3x	`.venv32`	Cliente OPC DA en Python (`OpenOPC.py`) [7]. No incluye `gbda_aut.dll` (§IV.3).
PySide6	`.venv`	Qt 6 para Python: ejecuta los archivos QML del HMI [13].
aiohttp, paho-mqtt	`.venv`	Puente y gemelo digital (WebSocket y MQTT). No se usan si el gemelo está apagado.
pytest	`.venv`	Pruebas automáticas (sin PLC ni ventana).

Tabla 4: Paquetes de cada entorno.

Comprobación de que todo se importa (es el paso [3/4] de `instalar.bat`):

Listado 5: Prueba de importación de cada entorno.

```
.venv32\Scripts\python.exe -c "import zmq, msgpack, loguru, win32com, OpenOPC; print('gateway 32 bits:
    dependencias OK')"
.venv\Scripts\python.exe -c "import zmq, msgpack, loguru, aiohttp, PySide6.QtQuick; print('HMI 64 bits:
    dependencias OK')"
```

Si la primera falla con `No module named win32com` o con un error de `pywintypes`, completa la instalación de pywin32 una vez:

Listado 6: Postinstalación de pywin32 en el entorno de 32 bits.

```
.venv32\Scripts\python.exe .venv32\Scripts\pywin32_postinstall.py -install
```

II.6. Instalación sin internet

Si la PC del laboratorio no tiene internet, descarga las ruedas en otra PC desde la raíz del repositorio, cópialas junto al repositorio e instala con `--no-index`. OpenOPC-Python3x se publica como código fuente, así que se descarga con `--no-binary`.

Listado 7: Descarga e instalación de ruedas para una PC sin internet.

```
rem En la PC con internet:
py -3 -m pip download --platform win32 --python-version 3.12 --only-binary=:all: -d ruedas32 pyzmq msgpack
    loguru pywin32
py -3 -m pip download --no-binary=:all: --no-deps -d ruedas32 OpenOPC-Python3x
py -3 -m pip download --platform win_amd64 --python-version 3.13 --only-binary=:all: -d ruedas64 -r
    requirements-hmi.txt pytest
rem En el laboratorio:
.venv32\Scripts\python.exe -m pip install --no-index --find-links ruedas32 -r requirements-gateway.txt
.venv\Scripts\python.exe -m pip install --no-index --find-links ruedas64 -r requirements-hmi.txt pytest
```

III. Qt: PySide6 y Qt Design Studio

III.1. PySide6 en el entorno del HMI

PySide6 se instala con `pip` dentro de `.venv` y ya trae Qt 6 completo; no hay que instalar Qt aparte para *ejecutar* el HMI. La versión se comprueba así:

Listado 8: Versión de PySide6 y de Qt.

```
.venv\Scripts\python.exe -c "import PySide6, PySide6.QtCore as c; print(PySide6.__version__, c.qVersion())"
6.11.2 6.11.2
```

La salida mostrada es la de la PC de desarrollo (9 de octubre de 2026); cualquier 6.8 o posterior sirve (`requirements-hmi.txt` pide `PySide6>=6.8,<7`). El código de `ui/` no importa PySide6 directamente, sino a través de `ui/qt_compat.py`, que elige el *binding* disponible:

Listado 9: Elección del binding de Qt (`ui/qt_compat.py`).

```
_PREFERIDO = os.environ.get("HMI_QT", "").strip().lower()
_ORDEN = ("pyqt5", "pyside6") if _PREFERIDO == "pyqt5" else ("pyside6", "pyqt5")
# Importa PySide6 si existe; si no, PyQt5 5.15. Los mismos QML corren en los dos.
```

Si en alguna PC sólo se pudiera instalar Python de 32 bits, el HMI corre con PyQt5, que sí tiene versión de 32 bits: `pip install PyQt5==5.15.11 PyQt5-Qt5==5.15.2` en ese intérprete y, si están los dos *bindings*, `set HMI_QT=PyQt5` antes de abrir el HMI.

Si la ventana del HMI sale negra o se cierra al instante, el problema es el OpenGL de la tarjeta de video. Se fuerza el dibujo por software antes de abrirlo:

Listado 10: Dibujo por software cuando falla OpenGL.

```
set QT_OPENGL=software
set QT_QUICK_BACKEND=software
lab\hmi_sim.bat
```

III.2. Qt Designer o Qt Design Studio

Qt tiene dos editores visuales y es fácil confundirlos. El HMI es 100 % QML (no tiene ningún archivo `.ui`), así que el que sirve es **Qt Design Studio** [14].

	Qt Designer	**Qt Design Studio**
Qué edita	Formularios `.ui` de Qt Widgets	Archivos `.qml` de Qt Quick y proyectos `.qmlproject`
Cómo se obtiene	Ya viene con PySide6: `.venv\Scripts\pyside6-designer.exe`	Instalador en línea de Qt, con cuenta de Qt (§III.3)
¿Sirve para este HMI?	No	Sí: abre `ui/qml/HMI.qmlproject`

Tabla 5: Diferencia entre los dos editores de Qt.

III.3. Instalar Qt Design Studio (opcional)

Qt Design Studio sólo hace falta en la PC donde se vaya a modificar la pantalla; el HMI se ejecuta sin él. En la PC de desarrollo se instaló la versión 4.8.3 en `E:\Qt`.

1. Crea una cuenta gratuita de Qt (la pide el instalador).

2. Descarga el instalador en línea, `qt-online-installer-windows-x64-online.exe` [15]. Qt Design Studio sólo se distribuye por este instalador.
3. Ejecútalo, inicia sesión y elige la carpeta de instalación (corta y sin espacios, por ejemplo `C:\Qt`).
4. En la selección de componentes marca **Qt Design Studio**. Las versiones de Qt 6 para C++ no son necesarias; si se dejan marcadas, la instalación crece a unos 8 GB (así quedó en la PC de desarrollo). Se pueden quitar después con `MaintenanceTool.exe`, en la misma carpeta.
5. El programa queda en `<carpeta>\Tools\QtDesignStudio\bin\qtdesignstudio.exe`.

III.4. Abrir el HMI en Qt Design Studio

En Qt Design Studio: *File* → *Open File or Project...* → `ui\qml\HMI.qmlproject`. El archivo principal es `PantallaHmi.qml`, el mismo que usa el HMI real, así que lo que se cambie en el editor es lo que se ve en la aplicación (Figura 2).

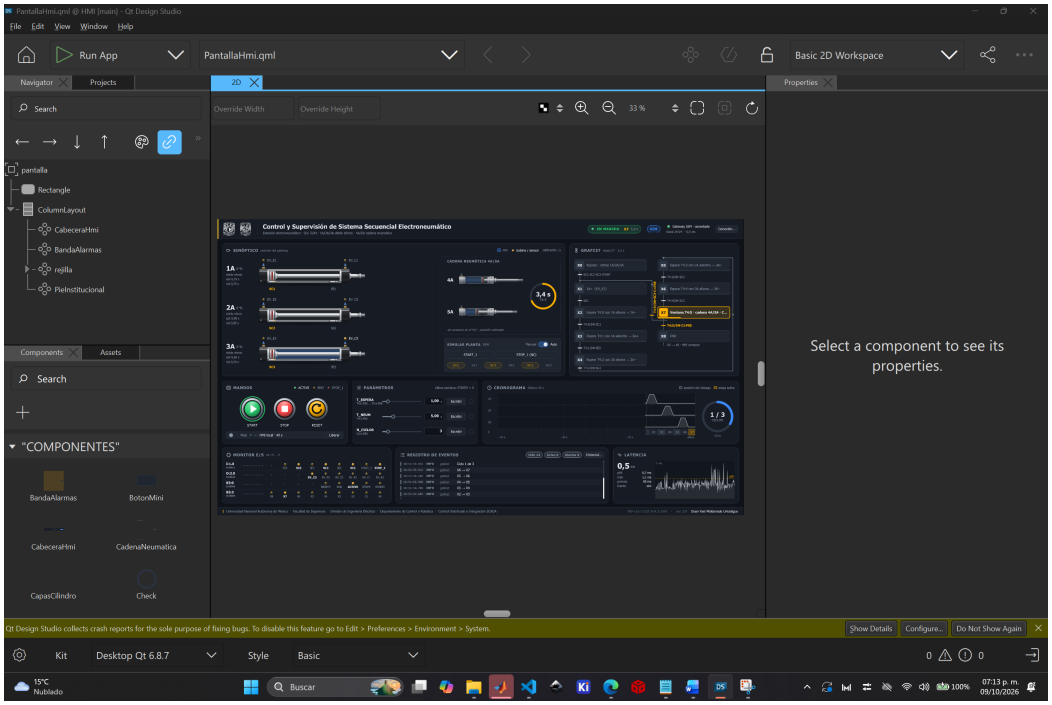


Figura 2: `PantallaHmi.qml` abierta en Qt Design Studio con los datos de prueba: a la izquierda el árbol de la pantalla y los componentes propios.

Listado 11: Proyecto de Qt Design Studio (`ui/qml/HMI.qmlproject`, extracto).

```
import QmlProject 1.1
Project {
    mainFile: "PantallaHmi.qml"
    mainUiFile: "PantallaHmi.qml"
    QmlFiles { directory: "." }
    Files { filter: "*.ttf;*.otf"; directory: "../../assets/assets_cilindros/fuentes" }
    importPaths: [ "." ]
    qt6Project: true
}
```

Qt Design Studio no ejecuta Python. Para que la pantalla tenga datos, carga los archivos de `ui/qml/dummydata/` (`planta`, `conexion`, `historial`, `datosInstitucion`, `rutaAssets`, `configVentana`), que imitan lo que pone `ui/app.py`. Esos archivos **se generan, no se editan**; se regeneran con el gateway simulado:

Listado 12: Regenerar los datos de prueba de Qt Design Studio.

```
.venv\Scripts\python.exe ui\herramientas\generar_dummydata.py
```

Cuidados al editar:

- No arrastres los paneles con el ratón: su posición sale de fórmulas de la rejilla (`x: rejilla.xCol(0)`) y arrastrarlos las cambia por números fijos, con lo que la pantalla deja de adaptarse de 1280×720 a 1920×1080. Colores, textos y propiedades de cada componente sí se pueden cambiar.
- La vista previa (*Run App*) usa los datos de prueba, no el PLC. El HMI real se abre con `lab\hmi.bat`.
- Qt Design Studio escribe preferencias locales (`HMI.qmlproject.qtds`); el `.gitignore` del HMI ya las excluye.

III.5. Prueba de la pantalla sin PLC

Antes de tocar RSLinx, ejecuta `lab\hmi_sim.bat`: abre el HMI junto con un gateway *simulado* (Ladder y física en Python). Debe verse el chip **SIM** azul en la cabecera, el LED de conexión verde, `Good 24/24` y el GRAFCET en X0 (Figura 3). Pulsa **START**: las etapas avanzan de X1 a X7, los cilindros se mueven y a los tres ciclos aparece «Fin de N ciclos (END)». Prueba también STOP, START y RESET. Al cerrar la ventana, el gateway simulado se cierra solo.



Figura 3: Pantalla principal del HMI.

IV. Conexión OPC DA con RSLinx

IV.1. La cadena de comunicación

Tramo	Configuración en el laboratorio
SLC 5/04 ↔ 1761-NET-ENI	Canal 0 del SLC, serie DF1 full duplex, 19 200 bps, CRC.
1761-NET-ENI ↔ RSLinx	Ethernet; driver `AB_ETH-2` hacia 192.168.1.80.
RSLinx (servidor OPC DA)	Servidor `RSLinx OPC Server` (o `RSLinx Local OPC Server`); topic `PROYECTO1_TEST` apuntando al SLC.
RSLinx ↔ Python	COM a través de `gbda_aut.dll` (ProgID `Graybox.OPC.DAWrapper`), pywin32 y OpenOPC, dentro del proceso de 32 bits del gateway.
Gateway ↔ HMI	ZeroMQ en `tcp://127.0.0.1:5555` (estado) y `5556` (órdenes).

Tabla 6: Tramos de la conexión entre el PLC y el HMI.

IV.2. Qué es `gbda_aut.dll` y por qué hace falta

Un servidor OPC DA, como el de RSLinx, expone *interfaces personalizadas* COM (`IOPCServer`, `IOPCItemMgt`, ...) pensadas para C++ [9]. Los lenguajes de *script* como Python, VBA o VBScript no las pueden usar directamente; usan la interfaz de *automatización* de OPC (*OPC Data Access Automation*, basada en `IDispatch`) [10]. Un *automation wrapper* traduce de una a otra. `gbda_aut.dll` es el de Graybox Software [8]: al registrarla, Windows conoce la clase `Graybox.OPC.DAWrapper`.

OpenOPC no trae su propio wrapper: al crear el cliente prueba, en orden, una lista de clases de automatización y usa la primera que esté registrada [5]:

Listado 13: Clases que prueba `OpenOPC.client()` (`OpenOPC.py`, paquete OpenOPC-Python3x 1.3.1, en dos renglones).

```
OPC_CLASS = ('Matrikon.OPC.Automation;Graybox.OPC.DAWrapper;HSCOPC.Automation;'
             'RSI.OPCAutomation;OPC.Automation')
# Se puede forzar una con la variable de entorno OPC_CLASS.
```

La DLL es *en proceso* y de 32 bits: se carga dentro del `python.exe` que crea el cliente. Por eso el gateway y la herramienta de diagnóstico corren con `.venv32`; desde el Python de 64 bits la clase no se puede crear (§V).

IV.3. Origen de la DLL incluida en esta carpeta

La copia de `instalacion_python\gbda_aut.dll` se tomó de **OpenOPC for Python** [5], que la distribuye en su carpeta `lib/` (también presente en la bifurcación de GitHub del mismo proyecto [6]). La DLL no es de OpenOPC, sino de Graybox Software; OpenOPC la incluye porque la necesita para hablar COM. Se deja en el repositorio porque el paquete que instala `pip`, **OpenOPC-Python3x 1.3.1, sólo contiene `OpenOPC.py`** y no trae la DLL [7]. (Las guías anteriores del HMI la buscan en `.venv32\Lib\site-packages\OpenOPC\lib`; con esa versión del paquete esa carpeta no existe.)

Propiedad	Valor
Producto	Graybox OPC Data Access Automation Wrapper
Fabricante	Graybox Software (© 2002–2008)
Versión del archivo	1.2.8.508 (producto 1.2.0.0)
Arquitectura	x86, 32 bits (encabezado PE, máquina `0x014C`)
Tamaño	282 624 bytes
SHA-256	`735500AC58A9CD1E6359094E8C0BB91E62F8F15677507DB8AEF2EAB592DFC283`
Clase COM que registra	`Graybox.OPC.DAWrapper`
Tomada de	OpenOPC for Python, carpeta `lib/` [5, 6]

Tabla 7: Datos de `gbda_aut.dll` (leídos de las propiedades del archivo).

Antes de registrarla, comprueba que el archivo es el mismo:

Listado 14: Huella SHA-256 de la DLL; debe coincidir con la Tabla 7.

```
certutil -hashfile instalacion_python\gbda_aut.dll SHA256
```

IV.4. Registrar la DLL

Se hace **una sola vez por PC** y no se pierde al reinstalar Python ni al borrar los entornos virtuales. Abre una consola **como administrador** (Inicio → escribe `cmd` → clic derecho → *Ejecutar como administrador*) y, desde la carpeta que contiene `instalacion_python`:

Listado 15: Copia y registro de `gbda_aut.dll` con el `regsvr32` de 32 bits.

```
copy instalacion_python\gbda_aut.dll C:\Windows\SysWOW64\
C:\Windows\SysWOW64\regsvr32.exe C:\Windows\SysWOW64\gbda_aut.dll
rem Para quitar el registro:
rem C:\Windows\SysWOW64\regsvr32.exe /u C:\Windows\SysWOW64\gbda_aut.dll
```

Debe aparecer el aviso «DllRegisterServer en C:\Windows\SysWOW64\gbda_aut.dll se realizó correctamente» [11]. Los nombres de las carpetas confunden: en Windows de 64 bits, `System32` contiene los programas de **64** bits y `SysWOW64` los de **32** bits. Si la DLL se copia a `System32`, un proceso de 32 bits que la busque en esa ruta es redirigido a `SysWOW64` [12], no la encuentra y el cliente falla con *Class not registered*. Por eso la DLL y el `regsvr32` van los dos de `SysWOW64`.

IV.5. Comprobar el registro desde Python

Listado 16: Prueba de la clase COM con el Python de 32 bits, desde la raíz del repositorio.

```
.venv32\Scripts\python.exe -c "import win32com.client as w; w.Dispatch('Graybox.OPC.DAWrapper'); print('
    Graybox.OPC.DAWrapper OK')"
```

Debe imprimir `Graybox.OPC.DAWrapper OK`. Si dice *Invalid class string* (*Cadena de clase no válida*), la DLL no está registrada; si dice *Class not registered*, se está usando un Python de 64 bits o la DLL quedó en otra carpeta.

IV.6. Preparar RSLinx

Elemento	En el laboratorio	Cómo se comprueba
Driver	`AB_ETH-2` hacia el 1761-NET-ENI en 192.168.1.80 (ya configurado). Sin ENI: driver RS-232 DF1 con *Auto-Configure*	*Communications* → *RSWho* muestra el 1747-L541 (Figura 4).
Topic	`PROYECTO1_TEST`, ya existe. En otra PC: *DDE/OPC* → *Topic Configuration* → *New*, mismo nombre, procesador SLC 5/04; *Polled Messages* 100 ms	*DDE/OPC* → *OPC Test Client*: el item `[PROYECTO1_TEST]I:1.0` con calidad *Good*.
OPC Core Components	Los instala RSLinx (si faltan, el *redistributable* x86 de la OPC Foundation)	`lab\probar_opc.bat --servidores` lista el servidor.
Programa del PLC	`PROYECTO1_TEST.RSS` en RUN: archivo B3 con 4 palabras, T4 con 6 temporizadores, `T4:0..4.PRE = 100`, `T4:5.PRE = 500`, `C5:0.PRE = 3`	RSLogix 500 en línea, *Data Files*.

Tabla 8: Configuración de RSLinx y del PLC [16].

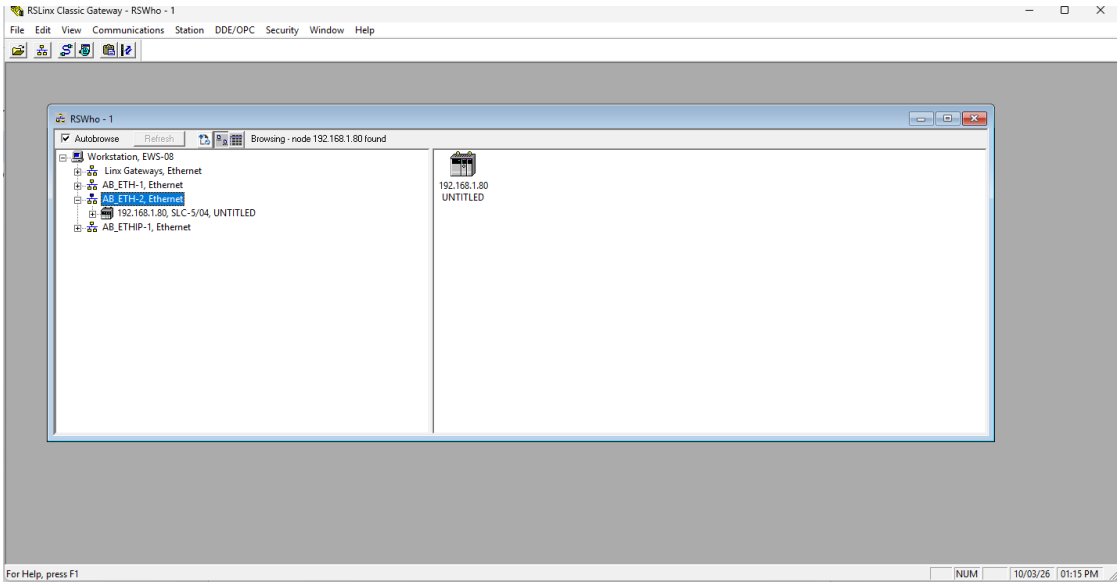


Figura 4: RSWho en la PC del laboratorio: el driver `AB_ETH-2` encuentra el SLC 5/04 a través del 1761-NET-ENI.

IV.7. Primera lectura con OpenOPC

Este programa mínimo prueba toda la cadena sin el gateway ni el HMI. Sólo lee; no escribe nada en el PLC. Guárdalo como `prueba_opc.py` en la raíz del repositorio y ejecútalo con el Python de 32 bits, con RSLinx abierto y el SLC en RUN:

Listado 17: `prueba_opc.py`: lectura de prueba por OPC DA. Uso: `.venv32\Scripts\python.exe prueba_opc.py`.

```
import OpenOPC

SERVIDOR = "RSLinx OPC Server"      # o "RSLinx Local OPC Server"
TOPIC = "PROYECTO1_TEST"

opc = OpenOPC.client()              # crea Graybox.OPC.DAWrapper (gbda_aut.dll)
print("Servidores:", opc.servers())
```

```
opc.connect(SERVIDOR)
print("Topics:", opc.list())

for direccion in ("I:1.0", "O:2.0", "B3:0", "T4:0.PRE"):
    item = "[%s]%s" % (TOPIC, direccion)        # p. ej. [PROYECTO1_TEST]I:1.0
    valor, calidad, hora = opc.read(item, sync=True)
    print("%-26s valor=%-6s calidad=%s" % (item, valor, calidad))

opc.close()
```

Debe imprimir la lista de servidores (con `RSLinx OPC Server` o `RSLinx Local OPC Server`), los topics (con `PROYECTO1_TEST`) y cuatro líneas con calidad `Good`; con el programa vigente, `T4:0.PRE` vale 100 (1,00 s). Si alguna sale `Bad`, revisa que el SLC esté en RUN y que el programa sea el de la tabla anterior.

IV.8. Diagnóstico con la herramienta del repositorio

`lab\probar_opc.bat` ejecuta `gateway.diagnostico_opc` con el Python de 32 bits. Hace lo mismo que el programa anterior y además mide tiempos:

Listado 18: Diagnóstico OPC paso a paso.

```
lab\probar_opc.bat --servidores
lab\probar_opc.bat --topics
lab\probar_opc.bat --topic PROYECTO1_TEST --modo REAL --n 30
lab\probar_opc.bat --topic PROYECTO1_TEST --modo REAL --probar-preajustes --confirmar-real
```

- `--servidores` y `--topics` deben listar el servidor de RSLinx y `PROYECTO1_TEST`.
- `--n 30` lee las cuatro palabras (`I:1.0`, `O:2.0`, `B3:0`, `B3:3`), los seis temporizadores y el contador, 30 veces desde el dispositivo y 30 desde la caché, con mínimo, media, p95 y máximo. Todo debe salir `Good`. Referencia del 3 de octubre: caché 6–8 ms por lectura; dispositivo 200–2238 ms.
- `--probar-preajustes` reescribe `T4:0.PRE` con su mismo valor (no cambia nada) y debe terminar en «OK, RSLinx acepta escribir PRE». El HMI escribe los parámetros con esa misma llamada.

IV.9. Cómo usa la conexión el gateway

El gateway abre el cliente igual que la prueba mínima, pero inicializa COM en su hilo y valida la tabla de direcciones contra el PLC antes de empezar:

Listado 19: Conexión en el gateway (`gateway/backend_opc.py`, `conectar()`).

```
import OpenOPC                  # solo Windows; dentro para que SIM no lo necesite
try:
    import pythoncom            # COM en el hilo actual (el del servidor)
    pythoncom.CoInitialize()
except Exception:
    pass
self.opc = OpenOPC.client()
self.opc.connect(self.servidor)
self._validar_direcciones()     # quita del sondeo lo que el PLC no tiene
```

Listado 20: Lectura del grupo rápido (`gateway/backend_opc.py`, `leer_rapido()`).

```
# "device" fuerza el viaje al PLC en cada llamada; "cache" devuelve lo último
# que RSLinx trajo con su propio sondeo, cuyo ritmo fija `update` (ms).
if self.fuente == "dispositivo":
```

```
    crudos = self.opc.read(self._items_rapidos, group=GRUPO_RAPIDO,
                           sync=True, source="device")
else:
    crudos = self.opc.read(self._items_rapidos, group=GRUPO_RAPIDO,
                           update=self.periodo_ms, sync=True, source="cache")
```

Los items siguen la sintaxis de RSLinx `[TOPIC]dirección`, por ejemplo `[PROYECTO1_TEST]I:1.0/3` o `[PROYECTO1_TEST]T4:5.PRE`. El grupo rápido tiene 22 items (cuatro palabras y `ACC`, `PRE` y `DN` de los seis temporizadores) y el lento dos (el contador): por eso la cabecera del HMI muestra `Good 24/24`. Reglas que el software aplica siempre en modo REAL: nunca escribe las salidas `O:2` ni las entradas `I:1`; START, STOP y RESET son pulsos de 300 ms sobre `B3:0/0`, `B3:0/1` y `B3:0/4`; y sólo acepta lecturas con calidad *Good*.

IV.10. Conectar el HMI en modo REAL

Con todo lo anterior en verde, abre `lab\hmi.bat`. Si no hay gateway, a los dos segundos se abre sola la ventana de conexión (Figura 5):

1. **Modo:** tarjeta **REAL**. Comprueba que de verdad quedó en REAL: en SIM los botones mueven la planta simulada y no llegan al PLC.
2. **Gateway:** *Arrancar aquí*. La línea *Intérprete* debe mostrar la ruta de `.venv32\Scripts\python.exe`; en rojo significa que falta el Python de 32 bits. Periodo 100 ms, fuente *caché*.
3. **OPC DA · RSLinx:** *Actualizar*, elegir el servidor, el topic `PROYECTO1_TEST`, un archivo y hacer doble clic en un item para leerlo (verde = *Good*).
4. **Envío de datos:** *Ninguno* (gemelo digital apagado).
5. **Conectar** y confirmar en el diálogo rojo. El pie debe decir **CONECTADO · modo REAL** y la cabecera `Good 24/24`.

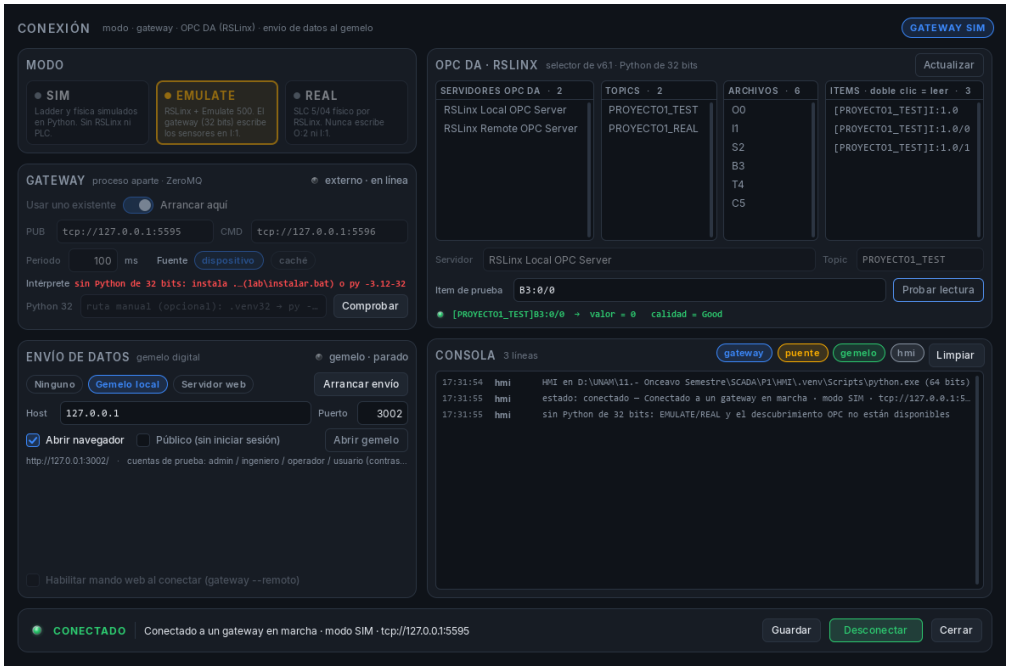


Figura 5: Ventana de conexión del HMI en la PC de desarrollo. Ahí no hay Python de 32 bits y la línea *Intérprete* sale en rojo; en la PC del laboratorio debe mostrar la ruta de `.venv32`.

V. Problemas típicos

Síntoma	Causa probable y solución
`py` no se reconoce, o `instalar.bat` no encuentra `py -3.12-32`	Falta el *py launcher* o el Python de 32 bits. Reinstalar con *Modify* y marcarlo.
`No module named win32com`	Postinstalación de pywin32 en `.venv32` (§II).
Invalid class string al crear el cliente	`gbda_aut.dll` no registrada. Registrar (§IV.4).
Class not registered	Python de 64 bits, o la DLL copiada a `System32` o registrada desde otra carpeta. Usar `.venv32` y `SysWOW64`.
`--servidores` no lista nada	RSLinx Lite (sin OPC), servicio de RSLinx detenido o Python de 64 bits.
Calidad *Bad* en todos los items	SLC fuera de RUN, topic hacia otro nodo, ENI sin red. Comprobar con el *OPC Test Client* de RSLinx.
Alarma «El PLC no tiene B3:3, T4:4. . . » o `Good 17/17`	El programa descargado no es `PROYECTO1_TEST.RSS`. Descargar el vigente y reconectar.
Bad en `B3` justo después de cerrar un gateway a la fuerza	Sus grupos siguen unos 6 min en RSLinx. Esperar o reiniciar RSLinx.
Ventana del HMI negra o se cierra al abrir	OpenGL: `set QT_OPENGL=software` y `set QT_QUICK_BACKEND=software`.
Qt Design Studio no muestra datos o imágenes	Regenerar `dummydata` (§III.3); abrir el `.qmlproject`, no el `.qml` suelto.

Tabla 9: Problemas frecuentes de la instalación.

VI. Lista de pruebas de la instalación

Marca cada prueba en el laboratorio y anota el resultado en la bitácora. Los comandos se escriben desde la raíz del repositorio.

ID	Prueba	Comando o acción	Resultado esperado	✔
P-01	Intérpretes	`py -0p`	Aparecen `-V:3.12-32` y `-V:3.13`	☐
P-02	Arquitectura	Listado de comprobación de 32 y 64 bits	Las dos líneas terminan en `True`	☐
P-03	Entorno del gateway	Importación en `.venv32`	`gateway 32 bits: dependencias OK`	☐
P-04	Entorno del HMI	Importación en `.venv`	`HMI 64 bits: dependencias OK`	☐
P-05	Pruebas automáticas	`lab\pruebas.bat`	Ninguna `failed` (212 `passed` en la PC de desarrollo)	☐
P-06	Pantalla sin PLC	`lab\hmi_sim.bat`	Chip SIM, `Good 24/24`, X0; START avanza X1…X7	☐
P-07	Integridad de la DLL	`certutil -hashfile ...`	SHA-256 igual a la Tabla 7	☐
P-08	Registro de la DLL	`regsvr32` de `SysWOW64`	«se realizó correctamente»	☐
P-09	Clase COM	`Dispatch('Graybox.OPC.DAWrapper')`	`Graybox.OPC.DAWrapper OK`	☐
P-10	Servidor y topic	`probar_opc.bat --servidores` y `--topics`	Servidor de RSLinx y `PROYECTO1_TEST`	☐
P-11	Lectura	`prueba_opc.py` o `probar_opc.bat --n 30`	Todo `Good`; caché de 6 a 8 ms	☐
P-12	Escritura	`probar_opc.bat --probar-preajustes --confirmar-real`	«OK, RSLinx acepta escribir PRE»	☐
P-13	HMI con el PLC	`lab\hmi.bat`, modo REAL	CONECTADO · modo REAL, `Good 24/24`; START mueve la planta	☐
P-14	Qt Design Studio (opcional)	Abrir `HMI.qmlproject`	Pantalla con datos de prueba, sin errores	☐

Tabla 10: Pruebas de aceptación de la PC del laboratorio.

Referencias

[1] Python Software Foundation, *Python Releases for Windows*, `https://www.python.org/downloads/windows/`.

[2] Python Software Foundation, *Using Python on Windows* (instalador y lanzador `py`), `https://docs.python.org/3/using/windows.html`.

[3] Python Software Foundation, *venv: Creation of virtual environments*, `https://docs.python.org/3/library/venv.html`.

[4] M. Hammond *et al.*, *pywin32: Python for Windows extensions*, `https://github.com/mhammond/pywin32`.

[5] B. Barnreiter, *OpenOPC for Python*, kit de herramientas OPC de código abierto (GPL v2); su carpeta `lib/` contiene `gbda_aut.dll`, `http://openopc.sourceforge.net/`.

[6] M. Kwiatkowski, *openopc* (bifurcación de OpenOPC for Python), repositorio con `lib/gbda_aut.dll`, `https://github.com/mkwiatkowski/openopc`.

[7] *OpenOPC-Python3x 1.3.1*, paquete de PyPI, `https://pypi.org/project/OpenOPC-Python3x/`.

[8] Graybox Software, *Graybox OPC DA Automation Wrapper*, `http://gray-box.net/daawrapper.php`.

[9] OPC Foundation, *Data Access Custom Interface Standard*, versión 2.05a.

[10] OPC Foundation, *Data Access Automation Interface Standard*, versión 2.02.

[11] Microsoft, *regsvr32*, `https://learn.microsoft.com/windows-server/administration/windows-commands/regsvr32`.

[12] Microsoft, *File System Redirector*, `https://learn.microsoft.com/windows/win32/winprog64/file-system-redirector`.

[13] The Qt Company, *Qt for Python (PySide6) documentation*, `https://doc.qt.io/qtforpython-6/`.

[14] The Qt Company, *Qt Design Studio Manual*, `https://doc.qt.io/qtdesignstudio/`.

[15] The Qt Company, *Qt Online Installer*, `https://download.qt.io/official_releases/online_installers/`.

[16] Rockwell Automation, *RSLinx Classic Getting Results Guide*, publ. LINX-GR001.

[17] D. Y. Maldonado Uriostigue, *SCADA-Digital-Twin* (HMI, gateway y gemelo digital), `https://github.com/DaanMaldonadoU/SCADA-Digital-Twin`; guías internas `docs/GUIA_INSTALACION_HMI.md` y `docs/DIAGNOSTICO_CONEXION_PLC.md`.